A Cluster-Local Softmax Merge Unit for RISC-V Vector Clusters

Anonymous Author(s)

***Abstract*—Blockwise attention updates a compact (m, l, O) state after every tile. A general-purpose core pays scalar control and normalization costs for every row, even when vector hardware handles the output update. This paper presents a cluster-local Softmax Merge Unit (SMU) for this recurrence. The SMU uses the existing memory-mapped control plane and TCDM scratchpad interface of a Spatz RISC-V vector cluster. It therefore leaves the ISA and the main vector pipeline unchanged. The datapath evaluates the maximum, two exponential scales, reciprocal-based weights, and the weighted output-vector update. Across 16 measured (N, D) coordinates, the full SMU outperforms an RTL-aligned scalar-plus-RVV baseline by 1.42–10.14×. Concurrent execution with a streaming core workload preserves the SMU execution time within 0.05% at $(16, 64)$, while the core slows by 0.6%. The results show that a compact auxiliary engine can remove per-row merge overhead without requiring an ISA extension.**

***Index Terms*—online softmax, attention, RISC-V vector processor, scratchpad, accelerator**

I. Introduction

Blockwise attention maintains an online normalizer and a partial output state instead of materializing a full attention matrix [1], [2]. Each completed tile merges one scalar maximum, one scalar normalizer, and one output vector into the running state. This recurrence has little instruction level parallelism across its scalar steps, yet it executes once for every row and tile. A vector core can update O, but it still pays the scalar merge, normalization, command, and synchronization costs.

We study whether a small cluster-local engine can execute this state update without changing the processor instruction path. Spatz combines compact RISC-V vector units with a banked L1 scratchpad [3]. Its cluster integration offers an opportunity: an auxiliary unit can receive commands through existing memory-mapped registers and exchange state through TCDM. This interface isolates the mechanism from the decoder, vector register file, and vector functional units.

This paper makes three contributions. First, it implements a full mixed-scalar online-softmax merge datapath behind an existing MMIO/TCDM boundary. Second, it evaluates the design against scalar and RVV-assisted software paths across a measured scaling matrix. Third, it measures the internal state-machine phases and shared-TCDM interference, which identify the remaining vector streaming bottleneck and quantify coexistence with a core workload.

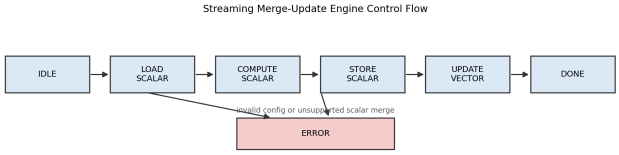


Fig. 1. The SMU controller loads scalar state, computes merge weights, and streams vector updates. Invalid configuration and unsupported scalar inputs terminate through the error path.

II. Online Merge and SMU Design

For an old state (m_o, l_o, O_o) and tile state (m_t, l_t, O_t), online softmax uses the recurrence

$$m_n = \max(m_o, m_t), \tag{1}$$

$$l_n = l_o e^{m_o - m_n} + l_t e^{m_t - m_n}, \tag{2}$$

$$O_n = \frac{O_o l_o e^{m_o - m_n} + O_t l_t e^{m_t - m_n}}{l_n}. \tag{3}$$

The scalar values select and normalize the vector update. The SMU streams these inputs directly from TCDM and writes m_n, l_n, and O_n back to TCDM.

Figure 1 summarizes the controller. The engine validates nonzero dimensions, aligned addresses, aligned strides, and supported finite scalar inputs before it starts a merge. The controller then loads four scalar values, computes the new maximum and scaled lengths, stores the scalar state, and streams the two output vectors. The engine reports explicit error status for invalid configurations and unsupported scalar inputs.

The implementation represents intermediate weights in fixed point. Two 256-segment linear LUTs approximate $\exp(x)$ over $[-8, 0]$ and a normalized reciprocal mantissa over $[1, 2]$. The reciprocal exponent restores scale after normalization. This organization confines nonlinear logic to the scalar path and keeps the vector loop to TCDM reads, weighted additions, and writes.

A. The interface defines a narrow offload contract

The software-visible contract consists of source and destination addresses for the six inputs and three outputs, dimensions N and D, a row stride, a start bit, and busy, done, and error status bits. A stride of zero selects packed rows with a default stride of $4D$ bytes. This convention lets software use contiguous state vectors without generating an explicit stride. The engine uses 64-bit TCDM transactions and packs or extracts one FP32 value according to the address half.

TABLE I
SMU COMMAND CONTRACT. ALL ADDRESSES IDENTIFY TCDM-RESIDENT FP32 ARRAYS OR SCALARS.

Field	Role
Source addresses	Old and tile m, l, and O state.
Destination addresses	Merged m, l, and O state.
N, D, stride	Row count, vector dimension, and vector row layout.
Start and clear	Launch one merge or clear terminal status.
Busy, done, error	Pollable progress and explicit validation result.

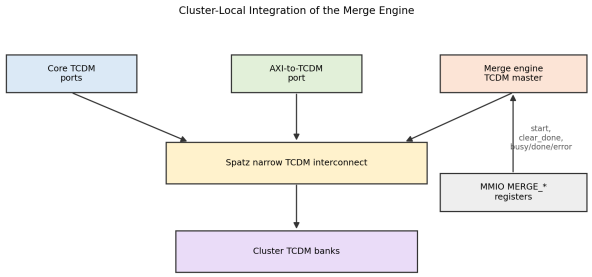


Fig. 2. Cluster integration. The SMU contributes one TCDM master and receives commands through the existing peripheral register path.

It therefore supports scalar arrays and packed vector arrays through one request format.

The controller uses five active states. `LOAD_SCALAR` issues four scalar reads. `COMPUTE_SCALAR` chooses m_n and forms scaled lengths. `COMPUTE_WEIGHT` applies the reciprocal approximation. `STORE_SCALAR` writes the two scalar outputs. `UPDATE_VECTOR` iterates over rows and elements, reads O_o and O_t, and writes the weighted sum. The engine accepts one operation at a time. This small FSM favors integration clarity and deterministic observability over aggressive pipelining.

B. The cluster integration avoids ISA changes

Figure 2 places the SMU beside the core complexes and their existing TCDM paths. The cluster peripheral owns the merge registers and forwards their values to the engine. The engine joins the TCDM interconnect as an additional requester. This organization avoids decoder entries, instruction encodings, register-file ports, and vector functional-unit changes. Software configures an operation, starts it, then observes terminal status through the same peripheral path.

III. EVALUATION

Selective scalar offloading retains the low vector cost of existing RVV and yields a 4.81× geometric-mean speedup in measured cycles over B2R. The Proposed design combines a Scalar SMU with existing RVV, while Full serves only as the Full-Offload Ablation. B2R supplies the head-to-head reference because it already performs the vector update with existing RVV. This reference makes scalar recurrence offload the measured question and keeps Full's role diagnostic. The primary result therefore concerns selective offload with the vector path held fixed in the comparison.

A. Experimental Setup

We evaluate FP32 state on a Spatz cluster with the (m, l, O) arrays resident in TCDM. The simulator runs RTL through Verilator, and every reported cycle count comes from the measured RTL/Verilator run. Each implementation uses the same generated state and command shape, so cycle differences reflect the execution path. The state includes old and tile scalar values together with output vectors, and the benchmark places source and destination arrays in TCDM before launch. We use one cluster configuration and one launch protocol across roles. RTL/Verilator cycles include the software-visible command and completion work, so each role exposes the path that its implementation actually executes.

We compare four roles. B1 runs the scalar software context. B2R runs the scalar recurrence in software and uses existing RVV for the vector update, making it the primary strong baseline. Proposed combines Scalar SMU with existing RVV. Full runs the Full-Offload Ablation, which moves the vector update into the SMU as well. We retain only outputs that pass the correctness gate; incomplete or failing cases do not enter a primary result. B1 supplies context for total scalar work but supplies no primary speedup. B2R keeps the vector loop on the existing RVV path, so its cycle count establishes the cost after vectorization. Proposed moves only the scalar recurrence across the SMU boundary, whereas Full moves both recurrence and vector update. This role assignment keeps each comparison tied to one architectural change.

We use Yosys/ABC with the Nangate45 library to collect standalone hardware evidence. An area-oriented `abc -fast` mapping reports formal cell counts and mapped area. A separate timing-driven mapping reports critical delay and an estimated standalone Fmax. The two flows answer different questions, so we keep their cell and area outputs separate from timing outputs. The available records provide no cluster PPA or power measurement. The area flow supports a standalone size comparison, while the timing flow supports a standalone timing estimate under its own constraints. We never combine a timing-flow cell count with the area-flow result. The evidence therefore supports the SMU-local comparison and leaves cluster-level PPA and power outside the reported scope.

B. Progressive Baseline

B1 to B2R isolates the contribution of RVV vectorization. B1 remains in Table II as cycle context, while B2R supplies the primary strong baseline for every speedup. This comparison identifies the cost that existing RVV removes before any SMU offload. The B1 column lets readers see how the scalar context grows with each model-derived shape, but it carries no speedup column. B2R preserves the same state representation and command setup while using the existing vector instructions, which makes the next comparison isolate the recurrence itself.

Every primary speedup divides Proposed cycles by B2R cycles. The ratio attributes improvement to scalar recurrence offload after RVV handles the vector update. Full appears only to quantify the cost and benefit of moving that vector update as an ablation. A common denominator keeps the interpretation stable across shapes and prevents a scalar-context comparison from becoming a headline result. The reported ratios thus measure the incremental contribution of Scalar SMU under the same RVV vector path.

C. Scaling and Cost Decomposition

To explain the cycle trend, we fit $C = C_0 + C_s N + C_v N D$ to each design. Here N denotes rows, D denotes vector dimension, C_0 captures fixed work, C_s captures per-row scalar recurrence work, and C_v captures per-element vector work. The fitted coefficients summarize RTL/Verilator cycle trends; they do not equal FSM busy-cycle counts. Figure 3 presents the fit and uses a logarithmic scale for C_s. The decomposition treats N and D as independent shape variables, so each coefficient summarizes a different growth direction. C_0 absorbs launch and loop-independent work, whereas C_s and C_v describe fitted contributions across the measured coordinates. None of these coefficients is a direct FSM state counter.

B2R fits $C_s/C_v = 1594.23/2.09$, whereas Proposed fits $90.28/2.13$. Proposed therefore cuts the scalar coefficient by $17.66\times$ while increasing the vector coefficient by only 1.88%. The near-constant C_v shows that Scalar SMU offload removes recurrence work after RVV vectorization without changing the existing vector path. The fitted scalar term accounts for the large difference between the two paths, while the fitted vector term stays within the same scale. This separation links the measured speedup to the specific recurrence work that Proposed moves off the core.

Full fits $C_s/C_v = 19.32/8.03$, and its C_v is $3.78\times$ Proposed's. Full therefore reduces the scalar coefficient further but pays a larger vector coefficient when it absorbs the vector update. The decomposition points to the selective tradeoff: low C_s with low C_v gives Proposed its measured advantage. Full quantifies the cost of extending scalar offload into the vector path, so the ablation separates the two sources of cycle change. Proposed keeps the lower vector coefficient while retaining the scalar reduction that matters for the primary baseline.

D. Model-Derived Configurations

We next evaluate one attention-position shape per model. For each shape, we generate a deterministic merge state with FP32 arrays in TCDM and measure the four roles under the same RTL/Verilator flow. The selected shapes are BERT $(N, D) = (12, 64)$, Mistral $(32, 128)$, and Qwen14B $(40, 128)$. Table II reports the resulting cycles, with B1 included as context. Each row represents one attention-position merge state, so the table exercises the recurrence at a model-derived (N, D) shape. Deterministic state generation keeps the inputs aligned across roles and makes the measured cycle columns directly comparable. The table reports merge cycles only and uses no end-to-end inference schedule.

Proposed reduces B2R cycles by $5.04\times$ for BERT, $4.62\times$ for Mistral, and $4.77\times$ for Qwen14B. Their geometric mean is $4.81\times$, matching the section result. These ratios keep the vector update in both compared paths and isolate scalar recurrence offload across model-derived dimensions. The three ratios remain close because each workload uses the same role definitions and the same vector path in Proposed and B2R. The geomean therefore summarizes a consistent selective-offload effect across the tested model dimensions rather than a single favorable shape.

Qwen72B's selected shape exceeds local TCDM capacity, so the benchmark has no cycle result for that shape and this boundary closes the measured feasible scope.

E. Hardware Cost and Full-Offload Ablation

Table III separates area evidence from timing evidence. The area flow uses formal cell count and mapped area from `abc -fast`; the timing flow uses a separate timing-driven mapping for critical delay and standalone Fmax. We keep timing-flow cell and area outputs out of the area comparison so the two mappings answer their intended questions. The table consequently reports formal cells and area from one flow, then delay and estimated Fmax from the other. This separation preserves the meaning of each column and prevents a timing result from changing the area comparison.

Proposed maps to 73,505 cells and 77,103.3 Liberty units with a 12.34 ns critical delay and an estimated standalone Fmax of 81.02 MHz. Full maps to 107,372 cells and 114,713.3 Liberty units with a 13.20 ns critical delay and an estimated standalone Fmax of 75.74 MHz. Full increases standalone mapped area by 48.8% over Proposed. The Proposed row therefore combines the smaller formal mapped design with the shorter timing-flow delay in this standalone evidence. Full exposes the additional mapped logic required when the vector update joins the offload path.

Figure 4 places B2R at $1.00\times$ speedup with zero SMU increment. Proposed reaches $4.81\times$ at 77.103 k mapped area, and Full reaches $2.00\times$ at 114.713 k. B2R's zero SMU increment denotes the software baseline and does not denote zero cluster area. The x-coordinate tracks standalone SMU mapped area, while the y-coordinate tracks measured cycles relative to B2R. The Full point shows how extra vector offload changes this standalone tradeoff, and workload crosses show variation across all three model-derived shapes in the measured comparison.

Using a common 81.0186 MHz assumption yields a $2.40\times$ Proposed/Full derived throughput ratio for the two standalone designs. The same frequency gives a $3.57\times$ incremental standalone-SMU area-efficiency ratio. These A1/A2-only derived comparisons do not measure system throughput or cluster efficiency. Cluster area, cluster timing and Fmax, physical area, signoff, power, and energy remain unavailable. The selective tradeoff is clear: Scalar SMU plus existing RVV preserves low vector cost while Full spends area to offload the vector update. The frequency assumption lets us compare the two standalone designs on one common scale, but it cannot turn

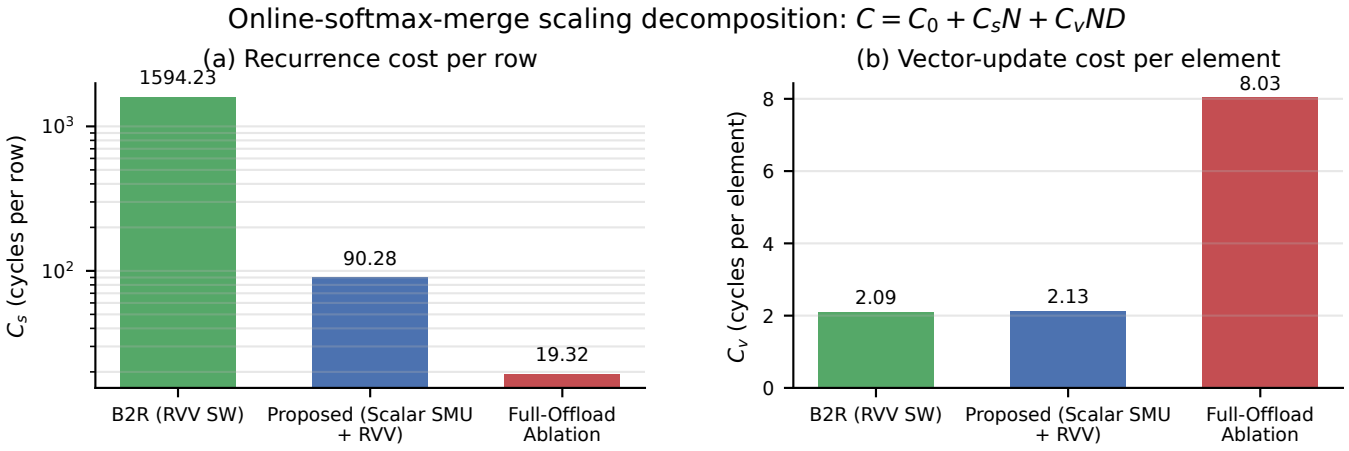


Fig. 3. Fitted C_s and C_v from RTL/Verilator cycles for B2R, Proposed, and Full. The C_s axis uses a log scale. The fitted terms describe cycle trends and exclude wall-time and FSM-busy-cycle interpretations.

TABLE II
MODEL-DERIVED ONE-POSITION SHAPES AND MEASURED RTL/VERILATOR CYCLES. VALUES DESCRIBE THE MERGE MICROBENCHMARK, NOT END-TO-END INFERENCE. B1 PROVIDES CONTEXT AND HAS NO SPEEDUP COLUMN.

Workload	(N, D)	B1 context	B2R	Proposed	Full ablation	Proposed/B2R
BERT	$(12, 64)$	271,718	20,785	4,128	7,704	5.04×
Mistral	$(32, 128)$	1,401,702	59,500	12,866	34,728	4.62×
Qwen14B	$(40, 128)$	1,780,547	75,027	15,733	43,206	4.77×
Geomean	–	–	–	–	–	4.81×

TABLE III
STANDALONE MAPPED HARDWARE EVIDENCE. THE AREA AND TIMING COLUMNS USE SEPARATE MAPPINGS; TIMING-FLOW CELLS AND AREA DO NOT ENTER THE AREA FLOW. FMAX IS A PRE-LAYOUT STANDALONE ESTIMATE.

Design	Mapped cells (area flow)	Mapped area (Liberty units)	Critical delay (timing flow)	Est. standalone Fmax
Proposed	73,505	77,103.3	12.34 ns	81.02 MHz
Full-Offload Ablation	107,372	114,713.3	13.20 ns	75.74 MHz

mapped evidence into a system measurement. The conclusion stays within A1/A2 evidence: selective scalar offload retains vector cost discipline while Full trades additional standalone area for vector-path offload in this ablation.

IV. DISCUSSION

A. The design targets a specific granularity

The SMU receives a complete merge command, so each launch carries a fixed configuration and completion cost. The measurements show that this choice works best when several rows amortize that cost. At $(1, 1)$, only 29 SMU busy cycles execute inside a 1,124-cycle end-to-end operation. At $(16, 64)$, the engine executes 8,534 busy cycles inside 9,556 end-to-end cycles. This change explains why the engine remains useful for matrix shapes that contain many short scalar recurrences, even when each recurrence has a compact vector.

The command boundary also defines a clear deployment path. A tiled attention kernel can retain its QK and PV computation on an existing vector or matrix engine, place the partial (m, l, O) state in local scratchpad memory, and submit merge commands at tile completion. The current prototype serializes one command, but independent rows and heads expose natural opportunities for multiple engines. A future front end can batch commands or use a queue to reduce the small-workload overhead that the current MMIO protocol reveals.

B. The evaluation distinguishes evidence layers

The paper uses three measurement layers because each answers a separate question. RTL end-to-end cycles establish the

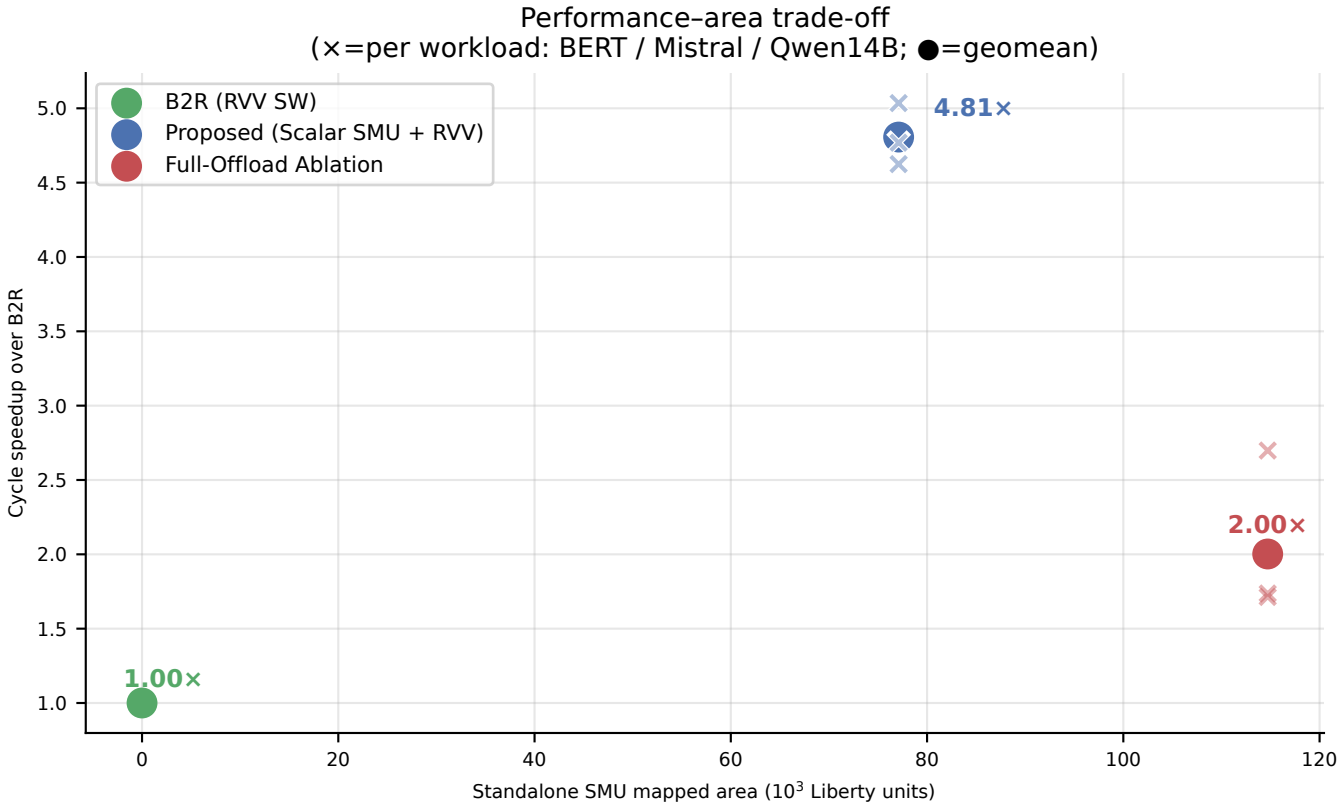


Fig. 4. Standalone SMU mapped area on the x-axis and frequency-independent measured-cycle speedup over B2R on the y-axis. B2R's zero SMU increment does not denote zero cluster area. Circles mark geomeans and crosses mark workloads; the plot makes no cluster-area claim.

software-versus-engine result. FSM counters establish where the engine spends active cycles. The concurrency experiment establishes whether a co-running stream alters those component times. The combination gives a stronger explanation than a speedup number alone: the scalar work leaves the core, the vector loop becomes dominant, and the shared scratchpad introduces measurable yet small interference for the tested stream.

The numerical results also have two layers. The host model tests the selected LUT interpolation and reciprocal construction against the complete recurrence. The RTL benchmark tests the implemented fixed-point data path against its RTL-aligned software reference. Reporting both values prevents the lower model error from becoming an end-to-end RTL claim. The explicit validation paths serve the same purpose. They show that the engine rejects malformed commands instead of silently producing an output.

C. Proxy data guides follow-up work

The generic logic and toggle measurements guide architectural choices but do not replace physical implementation. The resource decomposition assigns most generic cells to the vector arithmetic and data path, while the exponential and reciprocal blocks occupy smaller independent scopes. The toggle traces show a lower count for B3 at the largest sampled coordinate because B3 completes the operation in fewer target cycles. These observations motivate vector-path optimization and physical synthesis. They do not establish an area, frequency, power, or energy advantage.

V. Limitations and Related Work

Online normalization provides the recurrence that enables streaming softmax state [1]. IO-aware attention algorithms use this structure to process tiles without materializing full score matrices [2]. The SMU complements these algorithms at the cluster level. It accelerates the merge state after a tile; it does not implement QK multiplication, PV multiplication, masking, scheduling, or an end-to-end attention runtime.

Spatz supplies compact vector execution and a shared scratchpad [3]. Our design adds a TCDM master beside this execution path instead of introducing a vector instruction. The evaluation compares RTL-aligned software paths, not an optimized CPU, GPU, or production attention kernel. The Yosys and toggle numbers remain logic-activity proxies. A physical technology library, timing constraints, and power analysis would be required for PPA claims.

The approximation range also bounds the current result. The exponential LUT saturates below -8 and above zero, and the input validation excludes NaN, infinity, and subnormal values. The measured numerical cases cover mixed scalar inputs,

a delta sweep, and a merge-chain workload. They establish behavior for those cases; they do not provide a universal error bound over all attention distributions. A future design can add wider approximation ranges, refinement steps, or IEEE floating-point datapaths when the target workload requires them.

VI. Conclusion

The SMU moves online-softmax merge state updates into a compact TCDM-connected engine while preserving the Spatz ISA and vector pipeline. Measured B3 results outperform the RTL-aligned RVV-assisted baseline at all 16 scaling coordinates and retain component throughput during a shared-TCDM experiment. The measured FSM phases identify vector streaming and command overhead as the next design targets. These results establish cluster-local merge offload as a practical building block for future tiled-attention systems.

References

[1] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359.

[3] M. Cavalcante, M. Perotti, S. Riedel, and L. Benini, "Spatz: Clustering compact risc-v-based vector units to maximize computing efficiency," *arXiv preprint arXiv:2309.10137*, 2023.